

DSC 385 - Project 3 Report

Name and EID

Name: Marina Witherell

EID: mw44788

Setup

```
library(tidyverse)
library(tidymodels)
```

Warning: package 'tidymodels' was built under R version 4.5.3

Warning: package 'dials' was built under R version 4.5.3

Warning: package 'infer' was built under R version 4.5.3

Warning: package 'modeldata' was built under R version 4.5.3

Warning: package 'parsnip' was built under R version 4.5.3

Warning: package 'rsample' was built under R version 4.5.3

Warning: package 'tailor' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.3

Warning: package 'workflows' was built under R version 4.5.3

Warning: package 'workflowsets' was built under R version 4.5.3

Warning: package 'yardstick' was built under R version 4.5.3

```
## Load any additional packages that you need here
library(ranger)
```

Warning: package 'ranger' was built under R version 4.5.3

```
library(kknn)
```

Warning: package 'kknn' was built under R version 4.5.3

```
## You may want to use set.seed() to ensure that randomized results are
## reproducible across renders of your document
```

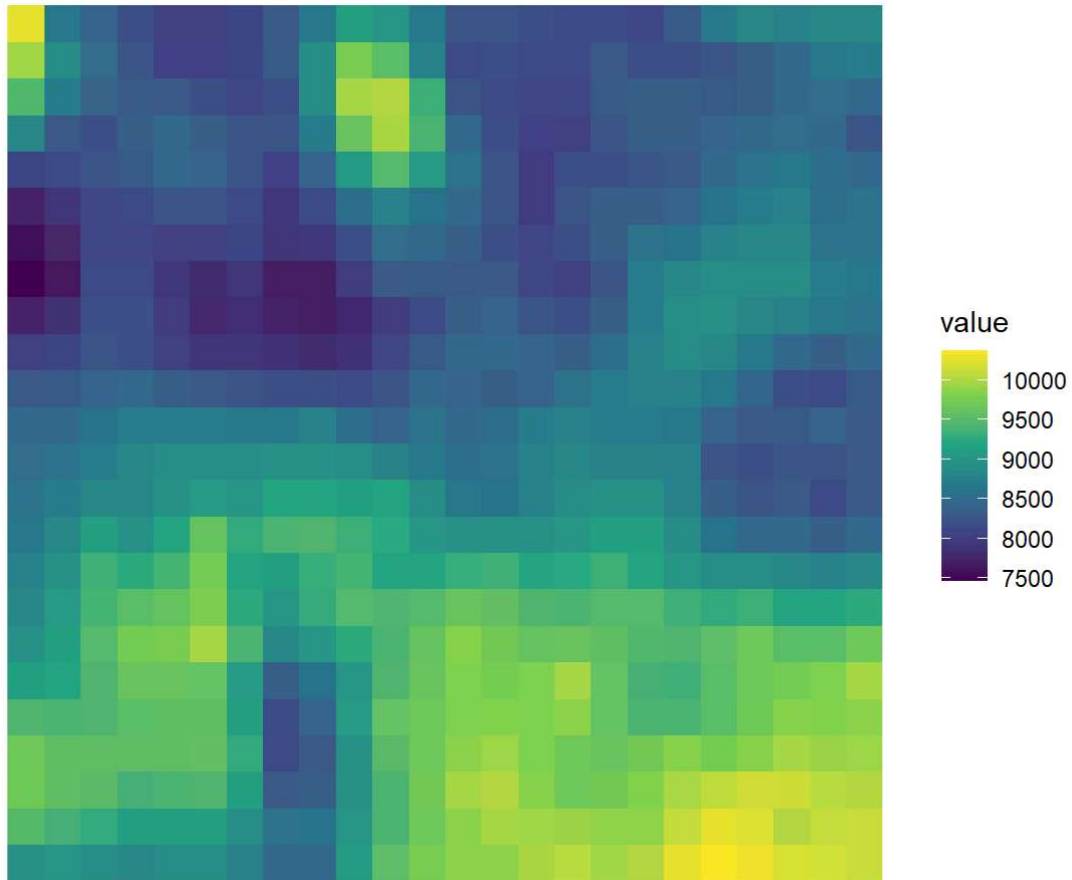
```
set.seed(123)
```

Read in the data with `read_csv()` and check the data to make sure all of the expected observations and variables are there.

```
## Read in the data
pm <- read_csv("pmFRM.csv.gz")
img_dat <- read_csv("image_data.csv.bz2")
```

The following annotated code is used to visualize a single image from a specific date and color band. You can feel free to modify it to visualize other dates or color bands. You can also delete this block if you want as it does not play a role in the analysis that follows.

```
img_dat |>
  ## Select a specific date and band
  filter(date == "2017-03-23",
         band == "band1") |>
  ## Select the pixel values
  select(value) |>
  ## Construct the grid/matrix corresponding to the pixel values
  bind_cols(expand_grid(col = 1:24, row = 1:24)) |>
  ## Setup the rows/columns of the image grid
  ggplot(aes(row, col)) +
  ## Fill the grid squares with pixel values
  geom_raster(aes(fill = value)) +
  ## Fix the aspect ratio to be 1:1
  coord_fixed() +
  ## Use Viridis color palette for pixel values
  scale_fill_viridis_c() +
  ## Remove background clutter
  theme_void()
```



Part 1: Prepare the data

The data in `img_dat` are in a long form and not suitable for use in the Tidymodels framework. The first thing you will need to do is to reshape the data using `pivot_wider()` so that there is one pixel/band combination per column. The data should look something like the following:

```
# A tibble: 1,217 × 2,305
  date      pixel001_band1 pixel001_band2 pixel001_band3 pixel001_band4
  <date>      <dbl>      <dbl>      <dbl>      <dbl>
1 2017-01-25      6185      5949      5631      4190
2 2017-02-08      3449      3374      3316      2254
3 2017-02-12      8630      7982      6938      4593
4 2017-02-13      6507      6250      5775      4244
5 2017-02-15      6706      6580      6137      4694
6 2017-02-19      6029      5907      5339      4069
7 2017-03-04      5643      5384      4532      3171
8 2017-03-15      9220      8468      7475      5631
9 2017-03-21      9074.      9062.      8644.      6881
10 2017-03-23      8949      8702      8387      6044
```

```
img_wide <- img_dat |>
  ## Reshape data
```

```

pivot_wider(
  names_from = c(pixel, band),
  values_from = value,
  names_sep = "_"
)
head(img_wide)

```

```

# A tibble: 6 × 2,305
  date      pixel001_band1 pixel001_band2 pixel001_band3 pixel001_band4
<date>      <dbl>         <dbl>         <dbl>         <dbl>
1 2017-01-25      6185           5949           5631           4190
2 2017-02-08      3449           3374           3316           2254
3 2017-02-12      8630           7982           6938           4593
4 2017-02-13      6507           6250           5775           4244
5 2017-02-15      6706           6580           6137           4694
6 2017-02-19      6029           5907           5339           4069
# i 2,300 more variables: pixel002_band1 <dbl>, pixel002_band2 <dbl>,
# pixel002_band3 <dbl>, pixel002_band4 <dbl>, pixel003_band1 <dbl>,
# pixel003_band2 <dbl>, pixel003_band3 <dbl>, pixel003_band4 <dbl>,
# pixel004_band1 <dbl>, pixel004_band2 <dbl>, pixel004_band3 <dbl>,
# pixel004_band4 <dbl>, pixel005_band1 <dbl>, pixel005_band2 <dbl>,
# pixel005_band3 <dbl>, pixel005_band4 <dbl>, pixel006_band1 <dbl>,
# pixel006_band2 <dbl>, pixel006_band3 <dbl>, pixel006_band4 <dbl>, ...

```

Next, you will need to join the data with the outcome data in the `pm` data frame.

```

full_dat <- pm |>
  ## Join datasets by date
  inner_join(img_wide, by="date")
head(full_dat)

```

```

# A tibble: 6 × 2,306
  date      pmFRM pixel001_band1 pixel001_band2 pixel001_band3 pixel001_band4
<date>      <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
1 2017-01-25  5.91           6185           5949           5631           4190
2 2017-02-08  7.93           3449           3374           3316           2254
3 2017-02-12  3.91           8630           7982           6938           4593
4 2017-02-13  3.93           6507           6250           5775           4244
5 2017-02-15  5.75           6706           6580           6137           4694
6 2017-02-19  3.75           6029           5907           5339           4069
# i 2,300 more variables: pixel002_band1 <dbl>, pixel002_band2 <dbl>,
# pixel002_band3 <dbl>, pixel002_band4 <dbl>, pixel003_band1 <dbl>,
# pixel003_band2 <dbl>, pixel003_band3 <dbl>, pixel003_band4 <dbl>,
# pixel004_band1 <dbl>, pixel004_band2 <dbl>, pixel004_band3 <dbl>,
# pixel004_band4 <dbl>, pixel005_band1 <dbl>, pixel005_band2 <dbl>,
# pixel005_band3 <dbl>, pixel005_band4 <dbl>, pixel006_band1 <dbl>,
# pixel006_band2 <dbl>, pixel006_band3 <dbl>, pixel006_band4 <dbl>, ...

```

Now split the data into training and testing datasets. Use 3/4 of the data for training and the remaining 1/4 for testing. Store the training data in a separate R object and leave the testing data aside for now.

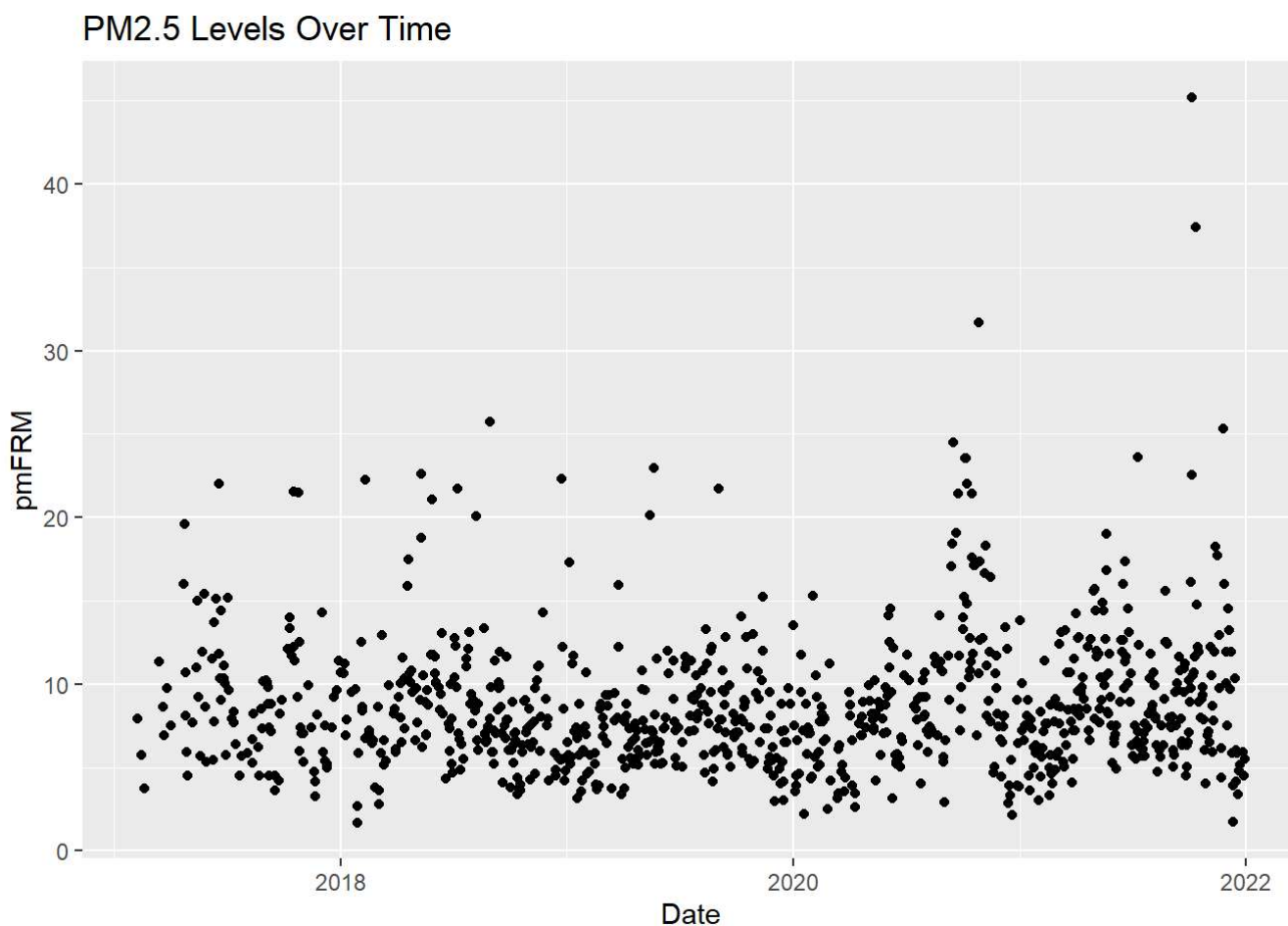
```
## Split the data
dat_split <- initial_split(full_dat)
## Training dataset
dat_train <- training(dat_split)
## Testing dataset (save for later)
dat_test <- testing(dat_split)
```

With the training data, visualize the ground-level monitor data by making a scatterplot of the `pmFRM` variable vs. `date`.

Describe any characteristics of the data that

1. May be useful for developing a prediction model
2. May present a challenge for developing a prediction model

```
ggplot(dat_train, aes(x = date, y = pmFRM)) +
  geom_point() +
  labs(title = "PM2.5 Levels Over Time", y = "pmFRM", x = "Date")
```



Write your answer to the questions here.

1. A characteristic that might be useful when developing a model is that the data seems to oscillate between local minimums and maximums throughout each year (suggesting PM might display seasonality).
2. Some challenges are that this data seems to have outliers that may skew the data and have leverage on the model.

Part 2: Linear regression model and feature extraction

In order to get an initial sense of the data, first develop a linear regression model WITHOUT using any of the satellite image data. That means, the only predictor variable you will be able to use is the `date` variable. This means that you will need to derive features from the `date` variable that may be useful for predicting ground-level PM. For this part:

1. Derive at least 2 features based on the `date` variable for inclusion in your model.
2. Specify your new features using the `step_*` functions in the recipe specification. In particular, `step_mutate()` is used to create new variables.
3. Use `linear_reg()` for your model.
4. Create a workflow that assembles your recipe and your model specification.
5. Use 10-fold cross-validation to estimate the root mean squared error (RMSE) and the R-squared for your model in the training data.

Try to develop a model that has an **RMSE less than 4 or an R-squared greater than 0.05**. You may need to play with the feature extraction a little while developing your model.

NOTE: You may want to review [Lecture 6.2](#) and [Lecture 6.3](#), which cover the use of the `lubridate` package to manipulate dates and times.

```
set.seed(123)
## Create recipe with new features
rec <- recipe(pmFRM ~ date, data = dat_train) |>
  step_mutate(
    month = factor(month(date)),
    quarter = factor(quarter(date))
  ) |>
  step_rm(date)

## Use linear regression model
model <- linear_reg() |>
  set_engine("lm") |>
  set_mode("regression")

## Create workflow
wf <- workflow() |>
  add_recipe(rec) |>
```

```
add_model(model)
```

```
## 10-fold cross validation  
folds <- vfold_cv(dat_train, v=10)  
res <- wf |>  
  fit_resamples(resamples = folds)
```

→ A | warning: prediction from rank-deficient fit; consider predict(., rankdeficient="NA")

There were issues with some computations A: x1

There were issues with some computations A: x3

There were issues with some computations A: x6

There were issues with some computations A: x9

There were issues with some computations A: x10

```
res |>  
  collect_metrics()
```

```
# A tibble: 2 × 6  
  .metric .estimator   mean     n std_err .config  
  <chr>   <chr>       <dbl> <int>   <dbl> <chr>  
1 rmse    standard     3.98     10  0.257 pre0_mod0_post0  
2 rsq     standard     0.0821    10  0.0239 pre0_mod0_post0
```

Write your RMSE and R-squared here.

- RMSE: 3.97640622
- R-squared: 0.08207188

Part 3: Diagnose the model

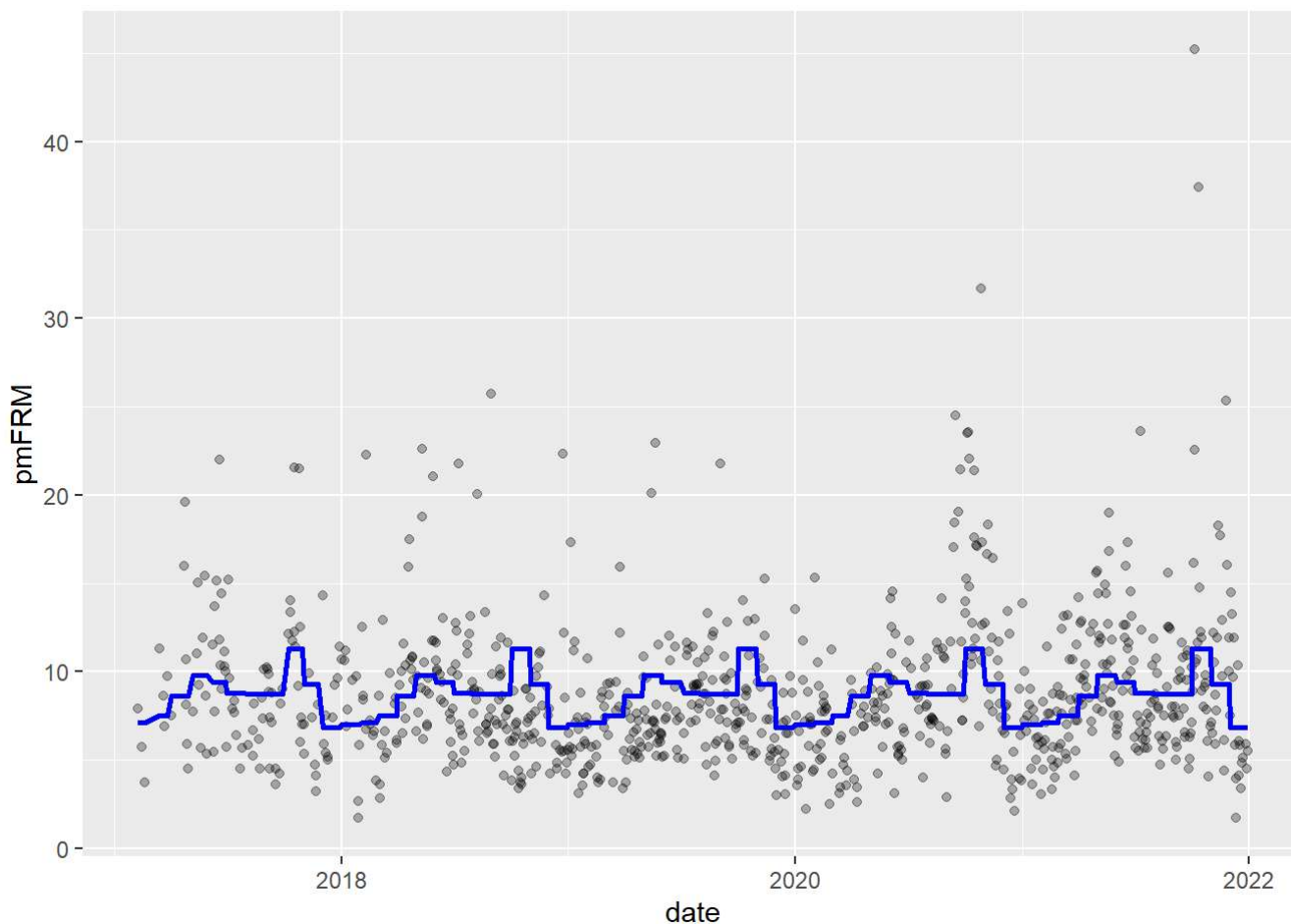
In this part we will diagnose the linear regression model from the previous section and identify possible ways that the model could be improved. Using the model you specified in the previous section, fit the model to the entire training dataset using the `fit()` function and store the result in an R object.

```
## Fit model to training dataset  
fitted <- wf |>  
  fit(data = dat_train)
```

Make a plot of the `pmFRM` vs. `date` variables and overlay the model predictions on the plot using `geom_line()`.

```
## Plot with predictions
fitted |>
  #extract_fit_parsnip() |>
  augment(dat_train) |>
  ggplot(aes(date)) +
  geom_point(aes(y=pmFRM), alpha=0.3) +
  geom_line(aes(y=.pred), color="blue", linewidth = 1)
```

Warning in predict.lm(object = object\$fit, newdata = new_data, type = "response", : prediction from rank-deficient fit; consider predict(., rankdeficient="NA")



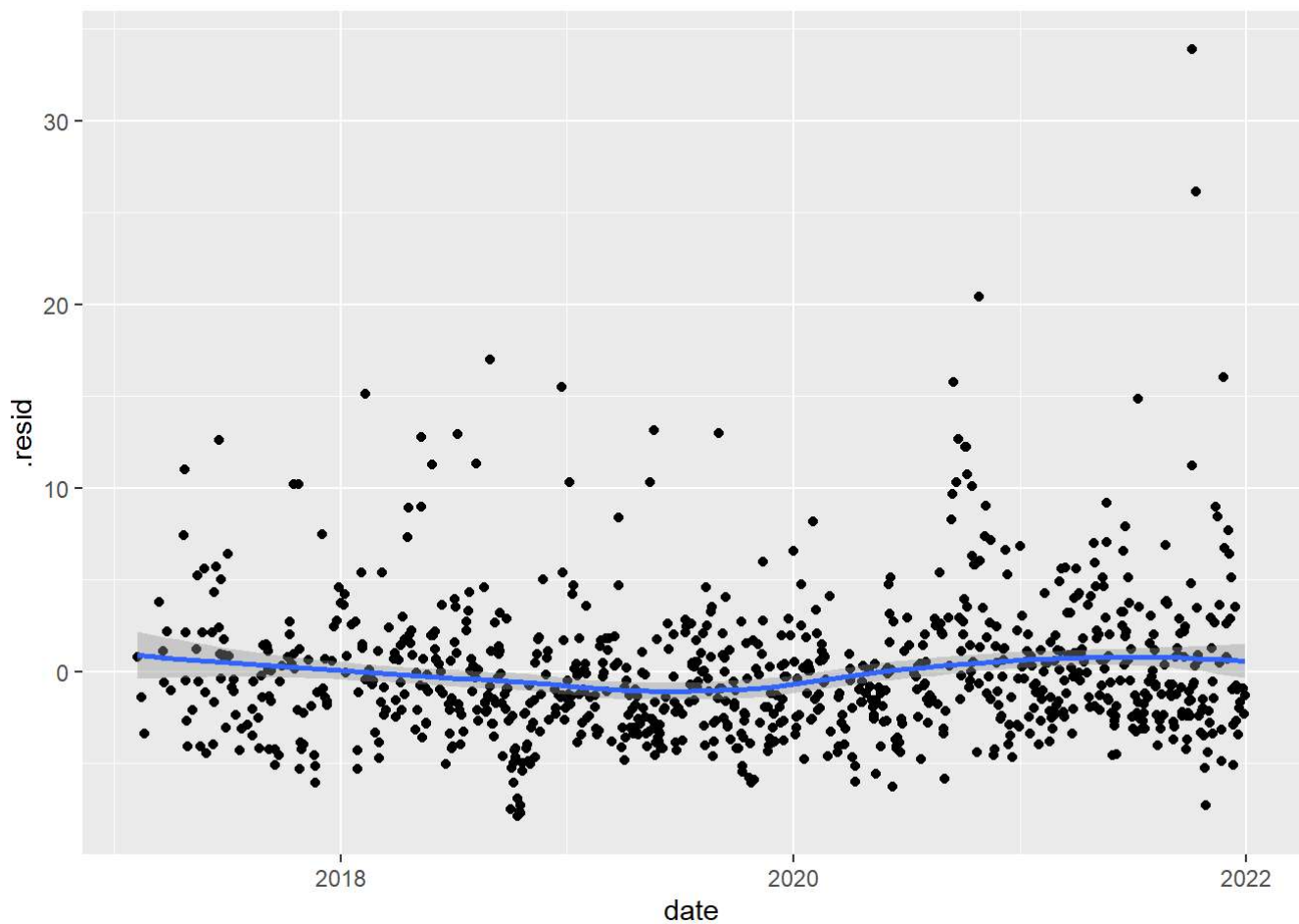
Make a plot of the residuals vs. date and overlay a smoother with `geom_smooth()` to detect any patterns in the residuals.

```
fitted |>
  augment(dat_train) |>
  ggplot(aes(date, .resid)) +
  geom_point() +
  geom_smooth()
```

Warning in predict.lm(object = object\$fit, newdata = new_data, type = "response", : prediction from rank-deficient fit; consider predict(.,


```
rankdeficient="NA")
```

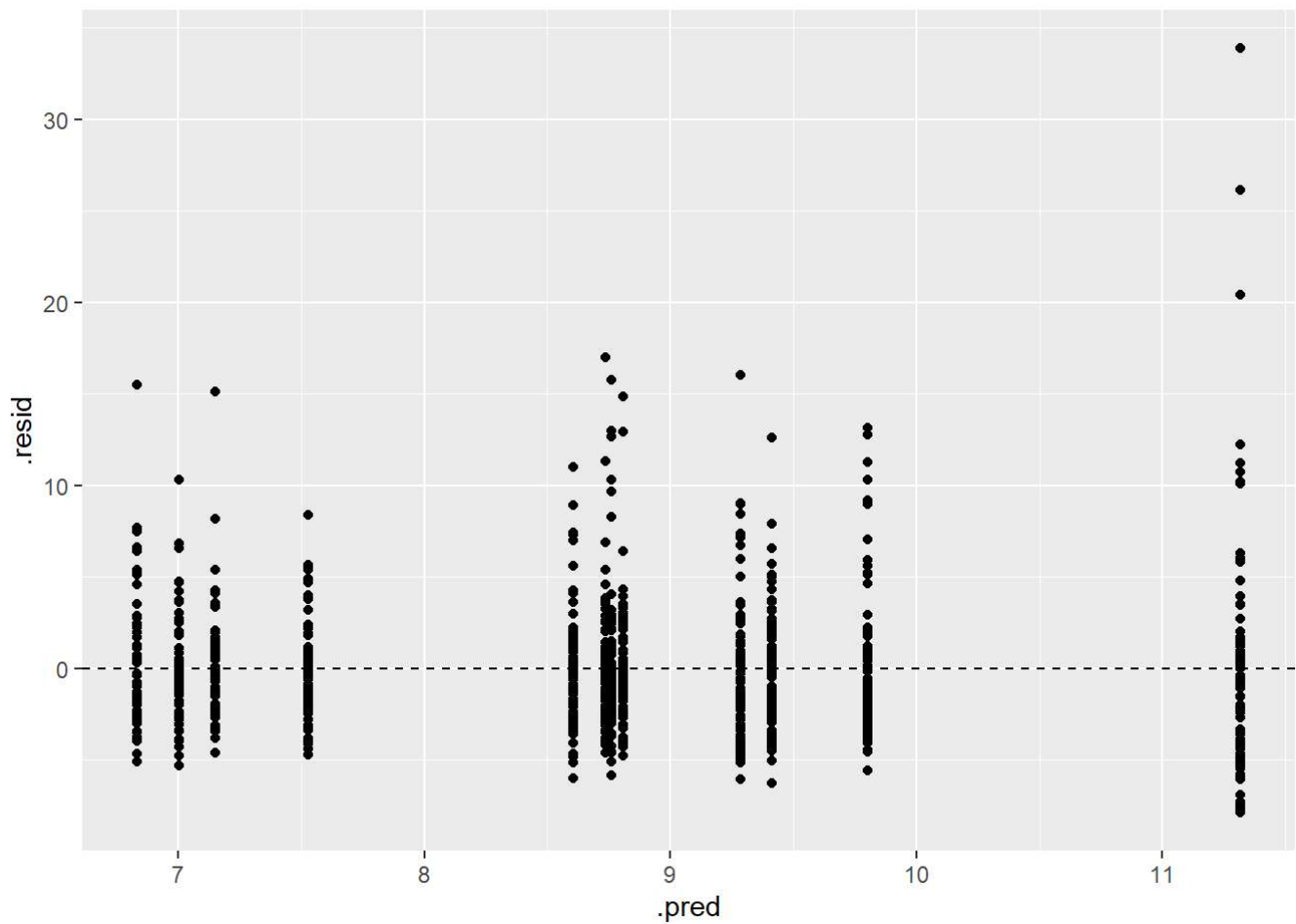
```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



Make a plot of the residuals vs. the predicted values.

```
fitted |>
  augment(dat_train) |>
  ggplot(aes(.pred,.resid)) +
  geom_point() +
  geom_hline(yintercept = 0,lty=2)
```

```
Warning in predict.lm(object = object$fit, newdata = new_data, type =
"response", : prediction from rank-deficient fit; consider predict(.,
rankdeficient="NA")
```

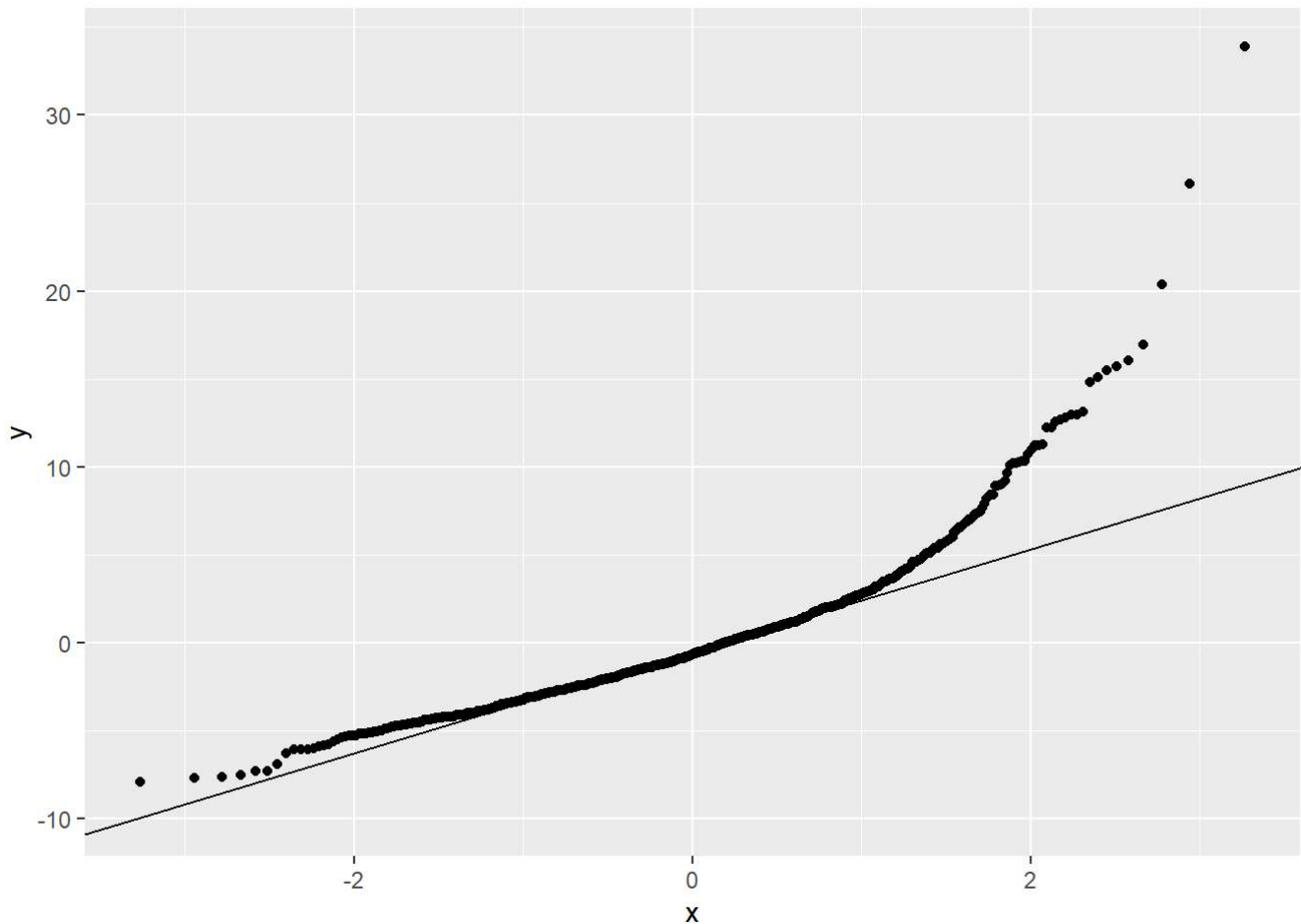


Make any other diagnostic plots or results that you think might be helpful to improve the performance of the model.

```
## Include additional diagnostic code here
```

```
fitted |>  
  augment(dat_train) |>  
  ggplot(aes(sample = .resid)) +  
  geom_qq() +  
  geom_qq_line()
```

```
Warning in predict.lm(object = object$fit, newdata = new_data, type =  
"response", : prediction from rank-deficient fit; consider predict(.,  
rankdeficient="NA")
```



Using the diagnostic plots and code you have produced above, describe at least two deficiencies of your linear model.

Write your answer here.

- The linearity assumption is not met since the points are not evenly scattered around the horizontal line in the residuals vs fitted plot (they display higher variance above the line).
- The normality assumption is not met since residuals are not normally distributed (they skew away from the QQ-line).

Part 4: Incorporate the image data

We will now incorporate the image pixel data and use a wider range of fitting algorithms to build a prediction model.

1. Choose a machine learning model for this part of the analysis. Ones we have seen already are `nearest_neighbor()`, `rand_forest()`, and `decision_tree()`. But feel free to use a different one if you wish. You can find a [list of possible models](#) on the Tidymodels web site. If you use a different model, you may need to install additional packages.
2. Identify the tuning parameters for your chosen model.

3. Specify a recipe using all the variables in the dataset
4. Create a workflow that assembles your recipe and your model specification.
5. Use 10-fold cross-validation to estimate the root mean squared error (RMSE) and the R-squared for your model in the training data.
6. Tune your model using a range of possible tuning parameters to identify the best-predicting model

Try to build a model that out-performs the linear regression model from the previous part.

NOTE: Doing the cross-validation and model tuning here may take a *significant* amount of time depending on your choice of tuning parameters and the speed of your computer. You may need to be patient with the process here.

```
set.seed(123)

## Machine Learning model
ml_model <- rand_forest(mtry = tune(), min_n = tune()) |>
  set_engine("ranger") |>
  set_mode("regression")

## Tuning parameters to test
#ml_grid <- grid_regular(
#  mtry(range = c(10,100)),
#  min_n(range = c(2,10)),
#  levels = 3)

## Recipe
ml_rec <- recipe(pmFRM ~ ., data = dat_train) |>
  step_mutate(
    month = factor(month(date)),
    quarter = factor(quarter(date)))

## Workflow
ml_wf <- workflow() |>
  add_recipe(ml_rec) |>
  add_model(ml_model)

## 10-fold cross validation and tuning
folds <- vfold_cv(dat_train, v=10)
ml_res <- tune_grid(
  ml_wf,
  resamples = folds,
  grid = expand_grid(mtry = c(10, 50, 100),
                    min_n = c(2,5))
ml_res |>
  collect_metrics()
```

A tibble: 12 × 8

	mtry	min_n	.metric	.estimator	mean	n	std_err	.config
	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
1	10	2	rmse	standard	4.06	10	0.171	pre0_mod1_post0
2	10	2	rsq	standard	0.0562	10	0.0142	pre0_mod1_post0
3	10	5	rmse	standard	4.07	10	0.170	pre0_mod2_post0
4	10	5	rsq	standard	0.0551	10	0.0141	pre0_mod2_post0
5	50	2	rmse	standard	4.03	10	0.170	pre0_mod3_post0
6	50	2	rsq	standard	0.0688	10	0.0192	pre0_mod3_post0
7	50	5	rmse	standard	4.02	10	0.173	pre0_mod4_post0
8	50	5	rsq	standard	0.0754	10	0.0210	pre0_mod4_post0
9	100	2	rmse	standard	4.01	10	0.171	pre0_mod5_post0
10	100	2	rsq	standard	0.0775	10	0.0192	pre0_mod5_post0
11	100	5	rmse	standard	4.01	10	0.175	pre0_mod6_post0
12	100	5	rsq	standard	0.0776	10	0.0198	pre0_mod6_post0

```
ml_res |>
  show_best(metric = "rmse")
```

A tibble: 5 × 8

	mtry	min_n	.metric	.estimator	mean	n	std_err	.config
	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
1	100	2	rmse	standard	4.01	10	0.171	pre0_mod5_post0
2	100	5	rmse	standard	4.01	10	0.175	pre0_mod6_post0
3	50	5	rmse	standard	4.02	10	0.173	pre0_mod4_post0
4	50	2	rmse	standard	4.03	10	0.170	pre0_mod3_post0
5	10	2	rmse	standard	4.06	10	0.171	pre0_mod1_post0

```
ml_res |>
  show_best(metric = "rsq")
```

A tibble: 5 × 8

	mtry	min_n	.metric	.estimator	mean	n	std_err	.config
	<dbl>	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
1	100	5	rsq	standard	0.0776	10	0.0198	pre0_mod6_post0
2	100	2	rsq	standard	0.0775	10	0.0192	pre0_mod5_post0
3	50	5	rsq	standard	0.0754	10	0.0210	pre0_mod4_post0
4	50	2	rsq	standard	0.0688	10	0.0192	pre0_mod3_post0
5	10	2	rsq	standard	0.0562	10	0.0142	pre0_mod1_post0

```
## Best model
best_rf <- select_best(ml_res, metric = "rsq")
best_wf <- finalize_workflow(ml_wf, best_rf)
```

Which one of your models (i.e. which combination of tuning parameters) produces the best prediction performance on the training dataset?

Write your answer here.

The model with the best prediction performance uses `mtry=100` and `min_n=5`.

Part 5: Diagnose the model

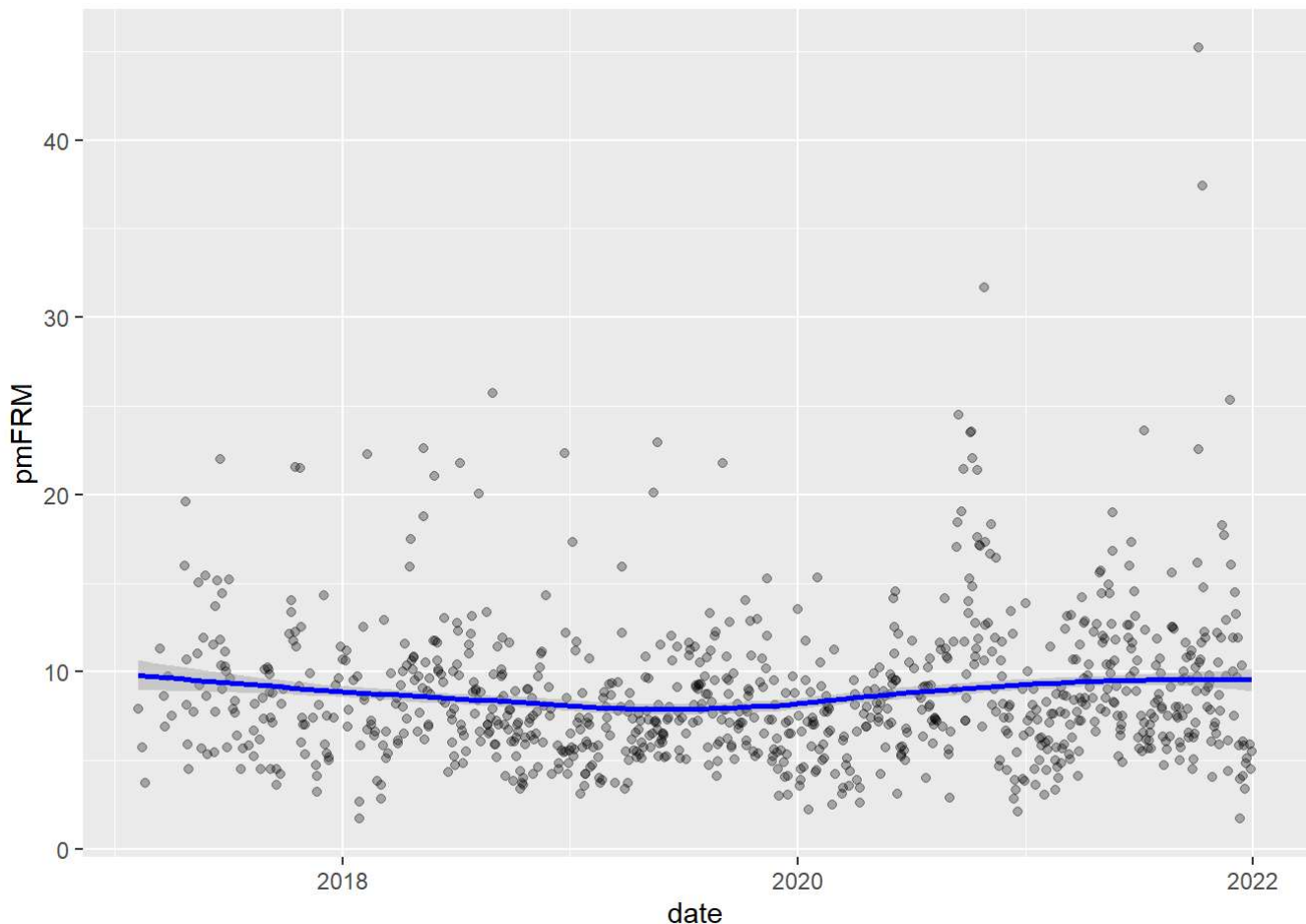
Fit your final model to the entire training dataset using the `fit()` function and diagnose the fit of your model using diagnostic plots or other diagnostic measures.

1. How does the performance of your final model compare with the performance of the linear model?
2. What shortcomings does your final model have that could be improved?

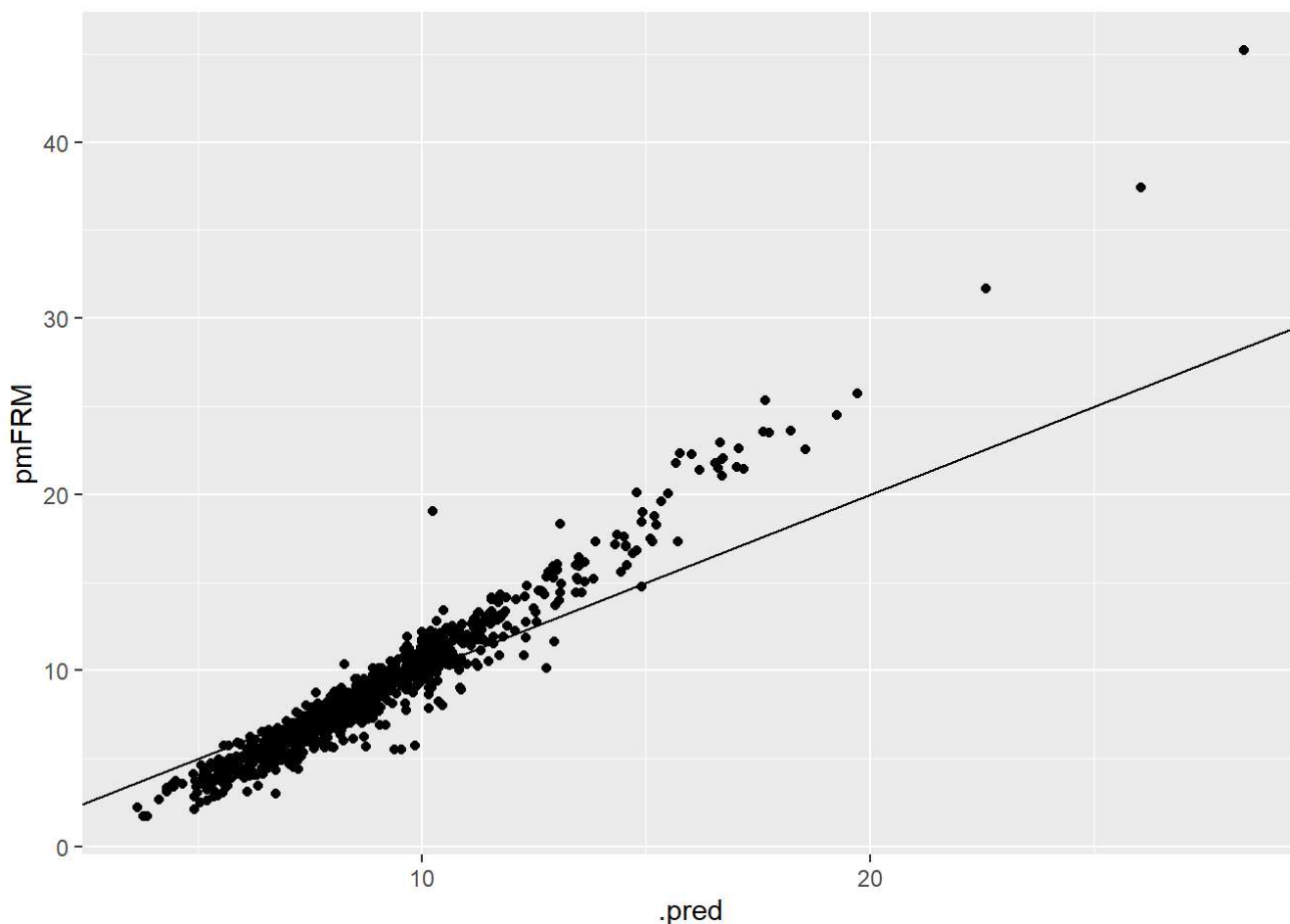
```
ml_fitted <- best_wf |>
  fit(data = dat_train)

ml_fitted |>
  augment(dat_train) |>
  ggplot(aes(date)) +
  geom_point(aes(y=pmFRM), alpha=0.3) +
  geom_smooth(aes(y=.pred), color="blue", linewidth = 1)
```

`geom_smooth()` using `method = 'loess'` and `formula = 'y ~ x'`



```
ml_fitted |>
  augment(dat_train) |>
  ggplot(aes(.pred, pmFRM)) +
  geom_point() +
  geom_abline()
```



Write your answer here.

1. The performance is slightly better, judging by the first graph. The line is slightly smoother than that of the one obtained using linear regression.
2. The model still does not seem to be a "perfect" prediction model for the data. The second graph shows that the data follows a straight line, but is not exactly on the line of predictions = pmFRM. This means that the predictions deviate with this model, especially as pmFRM increases.

Part 6: Discussion

Make a final assessment of your model using the testing dataset. What are the final RMSE and R-squared for your best model?

```
set.seed(123)

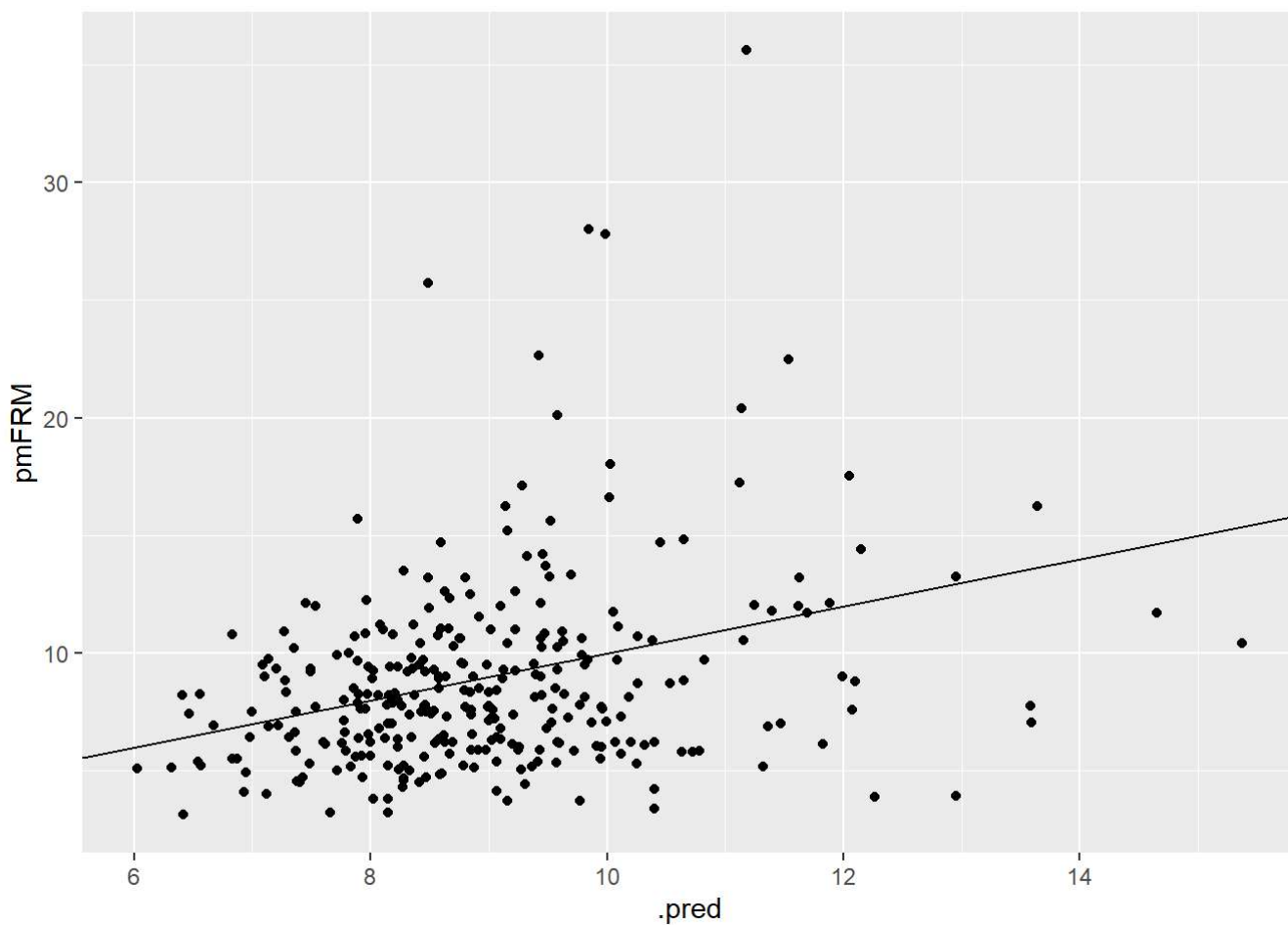
final <- best_wf |>
```

```
last_fit(split = dat_split)
final |>
  collect_metrics()
```

A tibble: 2 × 4

	.metric	.estimator	.estimate	.config
	<chr>	<chr>	<dbl>	<chr>
1	rmse	standard	3.92	pre0_mod0_post0
2	rsq	standard	0.0769	pre0_mod0_post0

```
final |>
  collect_predictions() |>
  ggplot(aes(.pred, pmFRM)) +
  geom_point() +
  geom_abline()
```



Write your answer here.

- Final RMSE: 3.92007705
- Final R-squared: 0.07694792
- Assessment: The RMSE is under 4 and the R-squared is over 0.05 in the testing dataset. This indicates that the model is a good predictor for the data and is not just memorizing the training set.

Reflect on the process of conducting this project. What was challenging, what have you learned from the process itself? Was there anything that was unexpected that you found during this analysis? If so, what was your expectation and how did the experience deviate from your expectation?

Write your answer here.

The most challenging aspect of this project for me was determining which machine learning model for prediction incorporating the image pixel data (in Part 4). I first tried using `rand_forest()` with 2x2 different tuning parameters. However, this did not yield results that performed better than the linear regression model. I then ran `nearest_neighbor()`, `decision_tree()`, and `xgboost()`. However, I ended up using `rand_forest()` with 3x2 tuning parameters for the final draft (shown in Part 4). I learned a lot about assessing different models and how to choose one that fits the data. Something that was unexpected was that there was no "perfect" model (at least not that I found) for the data. I am often used to there being a "right" answer in homework. However, having to choose the best model where there was no right answer was different than my expectation, but much more reflective of real-life scenarios.